MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Cuibari Vladislav FAF-243

Report

Laboratory Work No. 6

Operating Systems

Topic: The Command Line: Shell, Pipes and Text Filters

Checked by:

Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

INTRODUCTION 3
1 XV6 FILES AND DIRECTORIES 4
1.1 Directory entries and file types 4
2 MYSTAT AND HARD LINKS 5
2.1 The mystat implementation 5
2.2 Unlinking and the on-disk inode 6
3 FILE SYSTEM LIMITS 7
3.1 Name and file-size limits 7
4 RECURSIVE FIND 8
4.1 Recursive directory traversal 8
CONCLUSION 9
REFERENCES 10

INTRODUCTION

This laboratory studies xv6 file-system behavior from both its shell and its C source. I implemented mystat to inspect inode metadata and find to traverse directory entries recursively, then tested hard links, long names and the on-disk layout. The results below follow my saved xv6 terminal record; an initial reused disk image caused a directory-creation failure, so the final find test was run after make clean.

1 XV6 FILES AND DIRECTORIES

This chapter follows Part 1 of the handout. I decoded the columns of xv6 ls and inspected a directory already present in the file-system image.

1.1 Directory entries and file types

```
ls
mkdir a
ls a
```

The evidence for this exercise is shown in Figure 1.1.

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ ls
mkdir a
mkdir a
ls a.               1 1 1024
..              1 1 1024
README          2 2 2441
cat             2 3 36728
echo            2 4 35592
forktest        2 5 18136
grep            2 6 44184
init            2 7 35976
kill            2 8 35512
ln              2 9 35304
ls              2 10 42880
mkdir           2 11 35568
rm              2 12 35552
sh              2 13 58312
sleep           2 14 35280
args            2 15 35272
mystat          2 16 36320
find            2 17 43000
run             2 18 35888
pingpong        2 19 35872
run             2 18 35888
pingpong        2 19 35872
stressfs        2 22 36416
usertests       2 23 206800
grind           2 24 51824
wc              2 25 37728
zombie          2 26 34848
logstress       2 27 37560
forphan         2 28 36352
dorphan         2 29 35800
sync            2 30 34944
console         3 31 0
$ $ mkdir: a failed to create
```

Figure 1.1 – The xv6 root and an existing directory

Comment and observation

kernel/stat.h maps type 1 to directory, type 2 to file and type 3 to device. The root inode is 1, so root's '.' and '..' both refer to inode 1: the parent of root is root itself. An empty a is 32 bytes because it contains two 16-byte directory entries, '.' and '..' (a two-byte inode number plus a 14-byte name). mkdir a failed on the first boot because a was already present in the reused file-system image.

2 MYSTAT AND HARD LINKS

This chapter follows Part 2 of the handout. I implemented mystat, tested link counts before and after unlinking, and inspected the inode definitions in the xv6 source.

2.1 The mystat implementation

```
char *typename(int t) {
  if(t == T_DIR) return "directory";
  if(t == T_FILE) return "file";
  if(t == T_DEVICE) return "device";
  return "?";
}
for(int i = 1; i < argc; i++) {
  if(stat(argv[i], &st) < 0) {
    fprintf(2, "mystat: cannot stat %s\n", argv[i]);
    continue;
  }
  printf("%s: %s, inode %d, links %d, size %d bytes\n",
         argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
}
```

The program calls stat for each pathname and prints the type, inode number, hard-link count and byte size. A failed lookup reports to descriptor 2 and continues with later arguments. I added $U/_mystat to Makefile's UPROGS list; the build log shows user/_mystat was produced. One initial invocation printed 'exec mystat failed'; a repeated invocation produced the successful output below. The record does not establish the cause of the first failure.

```
mystat README . console
ln README r2
mystat README r2
rm README ; mystat r2
```

The evidence for this exercise is shown in Figure 2.1.

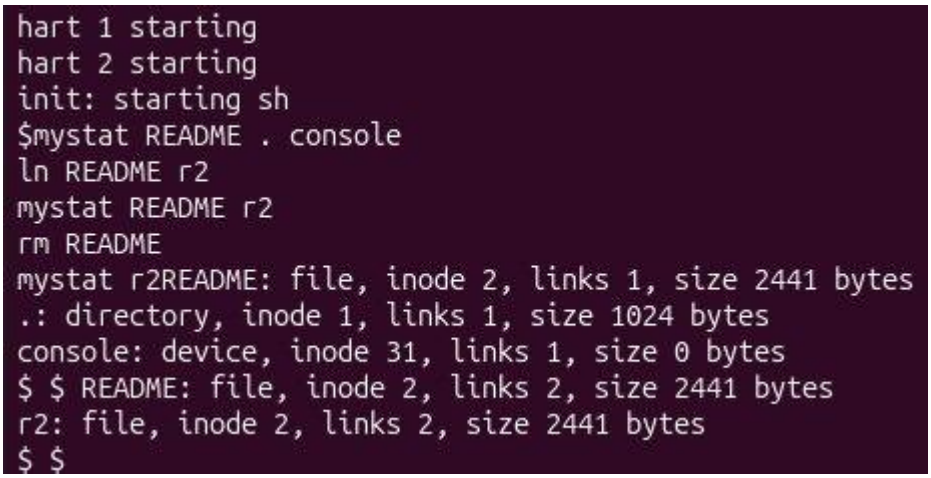


Figure 2.1 – mystat reports inode and link-count changes

Comment and observation

After ln, README and r2 both identified inode 2 and both showed two links. After rm README, r2 still identified inode 2 and its link count fell to one; the inode and 2441-byte contents did not change. cat r2 | wc returned 48 lines, 336 words and 2441 bytes. A separate lookup of nosuchfile correctly reported an error.

2.2 Unlinking and the on-disk inode

```
grep -n 'nlink' kernel/sysfile.c
grep -n -B2 -A7 'struct dinode' kernel/fs.h
```

The evidence for this exercise is shown in Figure 2.2.

```
dalvgame@dalvgame-VirtualBox:~/xv6-riscv$ cd ~/xv6-riscv
grep -n 'nlink' kernel/sysfile.c
grep -n -B2 -A7 'struct dinode' kernel/fs.h
145:  if (ip->nlink >= NLINK_MAX) {
151:  ip->nlink++;
158:  // dp may have been unlinked while we resolved it; linking into an
160:  // dropping ip->nlink).  create() has the same guard.
161:  if (dp->nlink == 0) {
178:  ip->nlink--;
202:sys_unlink(void)
220:  // Cannot unlink "." or "..".
228:  if (ip->nlink < 1)
229:    panic("unlink: nlink < 1");
237:    panic("unlink: writei");
239:    dp->nlink--;
244:  ip->nlink--;
269:  if (dp->nlink == 0) {
274:  // a new directory's ".." would push dp->nlink past its maximum
275:  if (type == T_DIR && dp->nlink >= NLINK_MAX) {
297:  ip->nlink = 1;
301:    // No ip->nlink++ for ".": avoid cyclic ref count.
311:    dp->nlink++; // for ".."
321:  ip->nlink = 0;
30-
31-// On-disk inode structure
32:struct dinode {
33-  short type;               // File type
34-  short major;              // Major device number (T_DEVICE only)
35-  short minor;              // Minor device number (T_DEVICE only)
36-  short nlink;              // Number of links to inode in file system
37-  uint size;                // Size of file (bytes)
38-  uint addrs[NDIRECT + 1]; // Data block addresses
39-};
40-
41-// Inodes per block.
42:#define IPB (BSIZE / sizeof(struct dinode))
43-
44-// Block containing inode i
45-#define IBLOCK(i, sb) ((i) / IPB + sb.inodestart)
46-
47-// Bitmap bits per block
48-#define BPB (BSIZE * 8)
49-
dalvgame@dalvgame-VirtualBox:~/xv6-riscv$
```

Figure 2.2 – Relevant source lines from the student's tree

Comment and observation

In sys_unlink, line 244 decrements the unlinked inode's nlink; line 239 separately adjusts a parent directory's count when a subdirectory is removed. The inode and data blocks can be reclaimed after no directory links and no live open references remain. The on-disk dinode stores type, major and minor device numbers, nlink, size and data-block addresses. This xv6 exercise has no Unix permission bits or symbolic-link type in that structure.

3 FILE SYSTEM LIMITS

This chapter follows Part 3 of the handout. I compared a long pathname with DIRSIZ, calculated the maximum file size and examined the mkfs disk layout.

3.1 Name and file-size limits

```
grep -n 'define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ' kernel/fs.h
mkdir averyveryverylongname ; ls
```

The evidence for this exercise is shown in Figure 3.1.

```
dalvgame@dalvgame-VirtualBox:~/xv6-riscv$ grep -n 'define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ' kernel/fs.h
5:#define BSIZE  1024 // block size
26:#define NDIRECT  12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
dalvgame@dalvgame-VirtualBox:~/xv6-riscv$
```

Figure 3.1 – Constants and truncated directory name

Comment and observation

ls showed averyveryveryl, the first 14 characters of the requested name; DIRSIZ is 14. With 12 direct blocks and 1024/4 = 256 indirect addresses, the largest file is (12 + 256) × 1024 = 274,432 bytes, or 268 KiB, in this single-indirect layout.

```
make clean ; make fs.img 2>&1 | grep nmeta
```

The evidence for this exercise is shown in Figure 3.2.

```
dalvgame@dalvgame-VirtualBox:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=
.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ mkdir averyveryverylongname
lsmkdir: averyveryverylongname failed to create
$
```

Figure 3.2 – mkfs disk layout

Comment and observation

The image totals 2000 blocks. Metadata uses 47 blocks: one boot block, one superblock, 31 log blocks, 13 inode blocks and one bitmap block. The remaining 1953 blocks are the data region. The long-name test and the later clean build were separate boots, so inode numbers can differ between them.

4 RECURSIVE FIND

This chapter follows Part 4 of the handout. I implemented recursive directory traversal and tested the optional inode-number output on a clean disk.

4.1 Recursive directory traversal

```
if(st.type == T_DIR) {
  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';
  while(read(fd, &de, sizeof(de)) == sizeof(de)) {
    if(de.inum == 0) continue;
    if(strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0) continue;
    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = 0;
    find(buf, name, showino);
  }
}
close(fd);
```

find opens a path, uses fstat to determine its type, compares the final pathname component with the target, and recursively reads nonempty directory entries. It skips '.' and '..' to avoid a cycle, checks buffer space, closes opened descriptors and accepts the optional -n flag for inode numbers. I registered $U/_find in Makefile; the build log shows user/_find was produced.

```
make clean ; make qemu
mkdir a
mkdir a/b
echo x > a/b/r2
echo y > a/r2
ln cat r2
find . r2
find a b
find . r2 -n
```

The evidence for this exercise is shown in Figure 4.1.



Figure 4.1 – Selected results from the clean-disk find test

Comment and observation

The clean-disk test found both nested r2 files and the root-level hard link. find a b returned a/b. With -n, ./r2 was inode 3, matching cat; the nested files had distinct inodes 30 and 31. An earlier echo failed, but the repeated command succeeded.

CONCLUSION

The implemented tools exposed xv6's compact file-system model. mystat showed that hard-link names share one inode while the link count changes. This connected the shell commands to the on-disk metadata studied in the source code.

Source constants explained the 14-character directory-name limit, the 268 KiB maximum file and the 2000-block disk layout. The clean-disk find tests showed recursive traversal, avoidance of '.' and '..', and identification of cat's hard link by inode number.

REFERENCES

OS 2026 Lab 6 handout, Inside the xv6 File System, provided for this course. Student terminal record. Source files consulted in the student's xv6 tree: kernel/stat.h, kernel/fs.h and kernel/sysfile.c.